The Platform: taking it to market

Written for you and Justin. It covers where the code should live, how Justin keeps control of fixes without it becoming a second full-time job, which AI tools to use, and how a new restaurant gets onto the platform.

---

1. Where the code lives

Short answer: export it to its own private GitHub repository now, keep hosting on Runable for the first clients, and give each environment its own database.

Piece	What it is today	What to do
Code	Bun + Vite + React 19 front end, Hono + oRPC API, Drizzle ORM, about 18,000 lines of TypeScript in `packages/web`	Push to a private repo, e.g. `jmrt39/blm-platform`. Justin is the owner; you are an admin.
Database	Turso (hosted SQLite), connected through `DATABASE_URL` + `DATABASE_AUTH_TOKEN`	One database for **production**, one for **staging**, and one for each developer locally. Never point a laptop at production.
File storage	S3-compatible bucket for log book photos and signatures (`S3_*` variables)	Separate bucket or prefix per environment.
Sign-in	Better Auth with email + password, wrapped by Runable's managed auth plugin	Fine while hosted on Runable. If you ever leave Runable, remove the `runableManagedAuth` plugin in `src/api/auth.ts`; the email + password sign-in keeps working on its own.
Hosting	Runable (the preview you've been using; publish from the platform UI)	Stay here for the first 3 to 5 clients. It's one click to publish, and the AI edit loop is already there.

When to move off Runable hosting: when you have more than about 10 groups, need a formal uptime agreement, or a client's IT asks for a SOC 2 style answer. The app is a plain Bun server (`bun src/__server.ts` after `vite build`), so it runs on **Fly.io, Railway or Render** with no rewrite. Budget about $20 to $50 a month at that size, plus Turso's paid tier.

Exporting

1. In Runable, use the project's export/GitHub option, or download the project and push it: `bash git init && git add . && git commit -m "Platform v1" git remote add origin git@github.com:jmrt39/blm-platform.git git push -u origin main`
2. **Never commit `.env`.** It holds the database token, auth secret and storage keys. Put the values in GitHub → Settings → Secrets and in the host's environment settings.
3. Rotate `BETTER_AUTH_SECRET` and the Turso token the first time you set up production outside Runable.

---

2. How Justin keeps control (without it eating his evenings)

Justin has a full-time job, so the aim is: **you can make small changes safely, and Justin reviews anything that touches data, money or sign-in.**

The rules

- **`main` is production.** Nobody pushes to it directly. Every change goes through a pull request (PR).

- **Branch per fix:** `fix/slow-night-undo`, `feat/pos-import`. Small PRs get reviewed in 10 minutes; big ones sit for a week.
- **Staging first.** Every PR deploys to staging (its own Turso database seeded with the demo group). Click through it, then merge.
- **Justin must approve** anything under `src/api/`, `src/api/database/schema.ts`, `src/api/auth.ts` or anything with money in it. Set this up with a `CODEOWNERS` file: `/packages/web/src/api/ @justin-github-handle` Copy and layout changes in `src/web/` can be approved by you.

The weekly loop

Who	What	Tool
You	Copy, labels, a new checklist line, a colour, "this button is confusing"	Runable chat on a **staging copy** of the project, or describe it in a GitHub issue
Justin	Bugs, data changes, new features	Clone the repo, branch, Claude Code or Cursor, open a PR
CI (automatic)	Lint, typecheck, build on every PR	GitHub Actions (below)
Sentry (automatic)	Emails Justin when something breaks in production, with the exact line	Sentry free tier

A realistic load once things settle is **2 to 4 hours a week** for Justin, mostly reviewing PRs and fixing what Sentry reports.

Continuous integration

Add `.github/workflows/ci.yml`:

```
name: CI
on: [pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v2
      - run: bun install --frozen-lockfile
      - run: bun run lint
      - run: cd packages/web && bunx tsc -p tsconfig.app.json --noEmit
      - run: bun run build:web
```

Make "CI passes" a required check on `main`.

Error tracking

Add **Sentry** (free up to 5,000 errors a month) to both the browser and the API. When a manager hits an error at 10pm, Justin gets an email the next morning with the user, the page and the stack trace. No guessing from a text message saying "it's broken".

Schema changes (the one thing to be careful with)

The database schema is in `packages/web/src/api/database/schema.ts`. Today it is pushed with `bun run db:push`, which is fine for a demo but risky with real client data.

- Before the first paying client: switch to migrations (`bun run db:generate`, commit the SQL file, then `bun run db:migrate` on deploy).
- Only add columns with defaults; never rename or drop a column in the same release that stops using it.
- Turso has point-in-time restore. Check it once, before you need it.

3. Which AI tools to use

Tool	Use it for	Cost (approx.)
Claude Code (Anthropic, in the terminal)	Justin's main tool. It reads the whole repo, runs the typecheck and lint, and makes multi-file changes. Best for "the slow night page double counts hours when you undo" type bugs.	Included with Claude Pro/Max, about $20 to $100 a month
Cursor	The editor, if Justin prefers to watch changes inline. Pick a Claude model inside it.	About $20 a month
Runable	Your tool for quick visual and copy changes, and for prototyping a feature before Justin hardens it. Work on a staging copy, then have Justin pull the change into a PR.	Current plan
GitHub Copilot code review (optional)	A second set of eyes on PRs	About $10 a month

How to prompt them well on this codebase. Point the tool at `design.md` and `task.md` first, and tell it the house rules:

- API features are one file each in `src/api/routes/`, added to the router in `src/api/index.ts`.
- Web data hooks live in `src/web/queries/<feature>.ts`.
- Never edit files whose names start with `__`.
- Every input needs a label.
- The rating stays 25% conduct / 75% performance.
- Anyone under 30 days is "ramping" and never ranked.

Then ask it to run lint and typecheck before it says it's finished.

What not to hand an AI unsupervised: the database schema, the sign-in setup, and anything that deletes data. Have it write the change, and Justin reads it line by line.

---

4. Onboarding a new restaurant (the skeleton)

A new group starts from a **blank organisation**: no fake data, only the standard library. Setup takes about an hour with the GM, and this is the flow:

1. **Owner signs up** and fills in one form: group name, first store, city, store PIN, dinner full time spend per ticket target (defaults to $78; lunch and part time targets sit alongside it). That creates the org, the store, and the standard opening and closing checklists.
2. **The home page shows a "Getting started" card** with seven steps. It ticks itself off as the data arrives and disappears when everything is done:
3. Add every store, with its PIN and target
4. Add the GM and managers
5. Import the front of house roster
6. Mark the trainers
7. Run the first opening checklist
8. Run the first line-up
9. Enter the first shift report from the POS
10. **Roster import** (`Team → Import roster`): paste or upload a CSV exported from 7shifts, HotSchedules, Homebase or the POS. Columns are name, role, hire date, email, phone and trainer. Roles can be written plainly ("Food Runner", "GM"), and dates can be `2025-03-11` or `3/11/2025`. You get a preview before anything is saved. Every person gets a personal PIN, shown on screen to print for line-up. Running it twice is safe because existing names are skipped.

11. **Hire dates drive the rules automatically.** Anyone under 30 days shows as ramping (not ranked, protected on slow nights), and anyone under 90 days gets a 30/60/90 plan.
12. **Already in place for every new org:** the 14 standard training modules, the knowledge checks and the grading rubrics. They're shared across the library, so nothing has to be copied in.

13. **The first week:** managers enter shift reports from the POS end-of-day screen. Spend per ticket comparisons start after 3 shifts per person, and the 30-day picture fills in from there.

14. **Before the group goes live, BLM builds their program in the Implementation Studio** (`/studio` , staff only):

15. **Brief:** paste the audit findings (AI suggests weighted priorities you edit), owner preferences, reputation notes and the spend per ticket target for each store.
16. **Materials:** paste or upload the menu, wine list, bar list, steps of service, standards, protocols and POS guide.
17. **Last year:** paste the POS daily export for a full previous year (date, net sales, checks, guests) per store. This drives "on this day last year" on Home, Reports and the line up.
18. **Build:** every BLM standard item (14 modules, 15 quizzes, 6 rubrics, 8 rewards, checklists, talking points) is listed. Adapt one or all with AI, edit, add new items (POS checks, repeat drills), then approve or exclude each.
19. **Publish:** only approved items go live. Republishing makes a new version and keeps staff progress.

Your one-day staff and manager training maps onto this: the morning is the floor (PIN sign-in, checklists, quizzes); the afternoon is managers (shift reports, line-up, conduct, slow night, reading the home page).

---

5. What v1 does, and does not, include

In v1 (live now): - Checklists with N/A and a reason - Shift reports - Spend per ticket by server, with ramping excluded - Line-up and pre-shift board - Conduct and write-ups - Training modules, knowledge checks and graded floor exercises - Updates with read receipts - Guest feedback, rewards and certifications - The log book with photos and signatures - The slow night planner - Multi-store view-as

Not in v1, on purpose:

Left out	Why	When
Scheduling	Every client already has 7shifts or HotSchedules, so you'd lose that fight. Import the roster from them instead.	Not planned
Payroll	Liability; leave it to the payroll provider	Not planned
Live POS feed	Toast and Square APIs need partner approval and per-client setup. Shift reports are entered by hand for now (about 3 minutes a night).	Year 2: Toast first, then Square
Billing	Year 1 is free with the engagement	Before the first year-2 renewal: Stripe subscriptions at $350 to $450 a month per store
Email and text notifications	Not needed while everything happens at line-up	When clients ask: Resend for email, Twilio for text
Back of house	Front of house only, by design	Only if a client pays for it

---

6. A checklist before the first paying client

- [ ] Code in a private GitHub repo; `.env` not committed
- [ ] Production and staging databases created; staging seeded with the demo

- [ ] CI required on `main`; CODEOWNERS for `src/api/`
- [ ] Sentry on the browser and the API
- [ ] Switch from `db:push` to migrations
- [ ] Turso point-in-time restore tested once
- [ ] A one-page privacy note for clients (what staff data is stored, who sees it, how to delete it)
- [ ] The client's GM has done the seven "Getting started" steps with you on day one